

PROGRAMAÇÃO DE BANCO DE DADOS

Prof. Rafael Tápio

ARQUIVO DA AULA DE HOJE - LINK

Unidade de Ensino 3

U3 - Seção 1

Aula

FOREIGN KEY E INTEGRIDADE REFERENCIAL

EXERCÍCIO

- Crie um banco de dados chamado **db_aula_20250327**.
- Use esse banco de dados para todos os exercícios da aula de hoje

```
CREATE DATABASE db_aula_20250327;  
USE db_aula_20250327;
```

FOREIGN KEY

CHAVE ESTRANGEIRA

Foreign Key (Chave Estrangeira)

Uma Foreign Key (ou Chave Estrangeira) é um campo em uma tabela que cria um vínculo com a chave primária de outra tabela.

Esse relacionamento é essencial para organizar e garantir a integridade dos dados em bancos de dados relacionais.

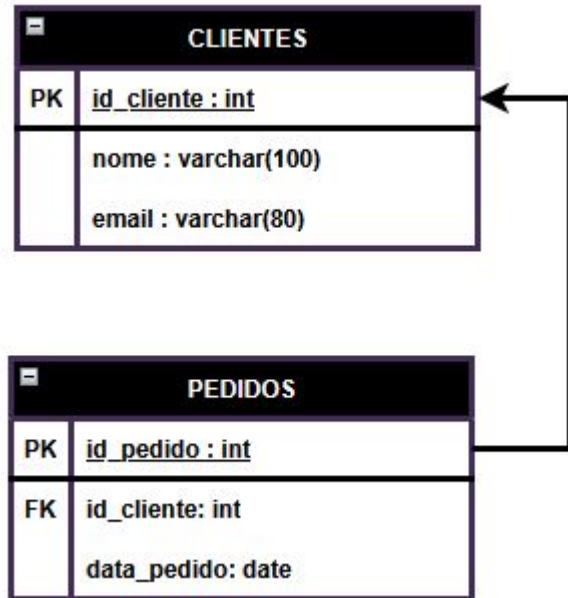
Exemplo Prático: Foreign Key

Imagine duas tabelas: **Clientes** e **Pedidos**.

- A tabela **Clientes** tem um campo chamado **id_cliente** (**chave primária**), que identifica cada cliente de forma única.
- A tabela **Pedidos** tem um campo **id_cliente** (**chave estrangeira**), que indica qual cliente fez o pedido.

A chave estrangeira **id_cliente** na tabela **Pedidos** faz referência à chave primária **id_cliente** na tabela **Clientes**. Isso estabelece uma relação entre as duas tabelas.

Exemplo Prático: Foreign Key




```
CREATE TABLE Clientes (  
    id_cliente INT PRIMARY KEY,  
    nome VARCHAR(100),  
    email VARCHAR(100)  
);
```

```
CREATE TABLE Pedidos (  
    id_pedido INT PRIMARY KEY,  
    id_cliente INT,  
    data_pedido DATE,  
  
    FOREIGN KEY (id_cliente)  
    REFERENCES Clientes(id_cliente)  
);
```

```
CREATE TABLE Pais(  
  id_pais INT PRIMARY KEY,  
  nome_pais VARCHAR(100)  
);
```

```
CREATE TABLE Estados(  
  id_estado INT PRIMARY KEY,  
  nome_estado VARCHAR(100),  
  id_pais INT,  
  
  FOREIGN KEY (id_pais) REFERENCES Pais(id_pais)  
);
```

```
CREATE TABLE Cidades(  
  id_cidade INT PRIMARY KEY,  
  nome_cidade VARCHAR(100),  
  id_estado INT,  
  
  FOREIGN KEY (id_estado) REFERENCES Estados(id_estado)  
);
```

Importância da Foreign Key

- **Integridade referencial:** Garante que os dados sejam consistentes, ou seja, não é possível inserir um pedido com um `id_cliente` que não existe na tabela de clientes.
- **Relacionamentos:** Facilita a criação de relações entre diferentes tabelas no banco de dados, tornando os dados mais organizados e interligados.

EXERCÍCIOS

Criando Tabelas de Autores e Livros

Crie um banco de dados com duas tabelas: **Autores** e **Livros**. A tabela Autores deve armazenar as informações dos autores, enquanto a tabela Livros deve armazenar os detalhes dos livros, incluindo o nome do livro e a data de publicação. A tabela Livros deve ter uma chave estrangeira que se relaciona com a tabela Autores. Insira ao menos 2 registros na tabela Autores, e 6 na tabela Livros

Passos:

- A tabela **Autores** deve ter **id_autor**, **nome_autor**, e **email_autor**.
- A tabela **Livros** deve ter **id_livro**, **titulo_livro**, **data_publicacao**, e uma chave estrangeira **id_autor** que faz referência à tabela **Autores**.
- Crie a chave primária para cada tabela e defina a Foreign Key corretamente na tabela Livros.
- Insira registros nas 2 tabelas

Sistema de Gestão de Cursos e Alunos

Crie um banco de dados simplificado para um sistema de gestão de cursos. O banco deve armazenar informações sobre **cursos, alunos e matrículas**.

Passos:

- A tabela **Cursos** deve ter id_curso, nome_curso.
- A tabela **Alunos** deve ter id_aluno, nome_aluno, e email_aluno.
- A tabela **Matrículas** deve associar **Alunos** a **Cursos**, contendo id_aluno e id_curso.
- Insira registros nas 3 tabelas

CONSISTÊNCIA DE DADOS

CASCADE, RESTRICT e SET NULL

Essas três opções estão relacionadas ao comportamento de exclusão de registros na **tabela filha** quando um registro na tabela pai é excluído. Elas são definidas durante a criação da chave estrangeira na tabela filha.

Esses comportamentos garantem a integridade referencial do banco de dados, ou seja, evitam dados órfãos (ou seja, registros na tabela filha que referenciam registros inexistentes na tabela pai).

CASCADE

Quando você define **ON DELETE CASCADE** na chave estrangeira, isso significa que, se um registro na tabela pai for excluído, todos os registros na tabela filha que referenciam esse registro serão excluídos automaticamente.

```
FOREIGN KEY (id_autor)  
REFERENCES Autores(id_autor) ON DELETE CASCADE
```

Exemplo:

Se um autor for excluído da tabela **Autores**, todos os livros que estão associados a esse autor serão excluídos da tabela **Livros**.

RESTRICT

Quando você define **ON DELETE RESTRICT**, a exclusão de um registro na tabela pai é restrita se houver registros na tabela filha que referenciam esse registro. Ou seja, não será permitido excluir o registro na tabela pai enquanto existirem registros na tabela filha que fazem referência a ele.

```
FOREIGN KEY (id_autor)  
REFERENCES Autores(id_autor) ON DELETE RESTRICT
```

Exemplo:

Se você tentar excluir um autor da tabela **Autores** enquanto ainda houver livros na tabela **Livros** relacionados a esse autor, a exclusão **será proibida e gerará um erro.**

SET NULL

Quando você define **ON DELETE SET NULL**, se um registro da tabela pai for excluído, a chave estrangeira da tabela filha que faz referência a esse registro será atualizada para NULL. Isso permite que os registros na tabela filha permaneçam, mas a referência ao registro excluído na tabela pai será apagada.

```
FOREIGN KEY (id_autor)  
REFERENCES Autores(id_autor) ON DELETE SET NULL
```

Exemplo:

Se um autor for excluído da tabela **Autores**, o campo **id_autor** dos livros na tabela **Livros** que estavam relacionados a esse autor será atualizado para **NULL**, ou seja, o livro não terá mais um autor associado.

COMO MODIFICAR UMA FOREIGN KEY?

Assim como as restrições, após a tabela já ter sido criada, é possível excluir e adicionar uma chave estrangeira, sem afetar os dados da tabela.

EXCLUIR

```
ALTER TABLE Livros  
DROP CONSTRAINT fk_autor_livros;
```

INCLUIR UMA FOREIGN KEY EM UMA TABELA EXISTENTE

```
ALTER TABLE Livros  
ADD CONSTRAINT fk_autor_livros  
FOREIGN KEY (id_autor)  
REFERENCES Autores(id_autor) ON DELETE CASCADE;
```

EXEMPLOS

CASCADE

Com o CASCADE, quando um autor é excluído, todos os livros desse autor serão excluídos também.

```
CREATE DATABASE db_exemplo_cascade;
USE db_exemplo_cascade;

-- Criando a tabela Autores
CREATE TABLE Autores (
    id_autor INT PRIMARY KEY,
    nome_autor VARCHAR(100)
);

-- Criando a tabela Livros com a chave estrangeira usando CASCADE
CREATE TABLE Livros (
    id_livro INT PRIMARY KEY,
    titulo_livro VARCHAR(100),
    id_autor INT,
    FOREIGN KEY (id_autor) REFERENCES Autores(id_autor) ON DELETE CASCADE -- CASCADE
);

-- Inserindo dados
INSERT INTO Autores (id_autor, nome_autor) VALUES (1, 'Machado de Assis');
INSERT INTO Autores (id_autor, nome_autor) VALUES (2, 'Chico Buarque');

INSERT INTO Livros (id_livro, titulo_livro, id_autor) VALUES (1, 'Dom Casmurro', 1);
INSERT INTO Livros (id_livro, titulo_livro, id_autor) VALUES (2, 'Senhora', 1);
INSERT INTO Livros (id_livro, titulo_livro, id_autor) VALUES (3, 'Budapeste', 2);

-- Verificando os dados antes do DELETE
SELECT * FROM Autores;
SELECT * FROM Livros;

-- Excluindo um autor (autor com id 1)
DELETE FROM Autores WHERE id_autor = 1;

-- Verificando os dados após o DELETE (Os livros relacionados ao autor excluído também serão excluídos)
SELECT * FROM Livros;
```

CASCADE

Autores	
id_autor	nome_autor
1	Machado de Assis
2	Chico Buarque

Livros		
id_livro	titulo_livro	id_autor
1	Dom Casmurro	1
2	Senhora	1
3	Budapeste	2

RESTRICT

Com o RESTRICT, não será permitido excluir um autor se houver livros relacionados a ele. A exclusão será bloqueada.

```
CREATE DATABASE db_exemplo_restric;
USE db_exemplo_restric;

-- Criando a tabela Autores
CREATE TABLE Autores (
    id_autor INT PRIMARY KEY,
    nome_autor VARCHAR(100)
);

-- Criando a tabela Livros com a chave estrangeira usando RESTRICT
CREATE TABLE Livros (
    id_livro INT PRIMARY KEY,
    titulo_livro VARCHAR(100),
    id_autor INT,
    FOREIGN KEY (id_autor) REFERENCES Autores(id_autor) ON DELETE RESTRICT -- RESTRICT
);

-- Inserindo dados
INSERT INTO Autores (id_autor, nome_autor) VALUES (1, 'Machado de Assis');
INSERT INTO Autores (id_autor, nome_autor) VALUES (2, 'Chico Buarque');

INSERT INTO Livros (id_livro, titulo_livro, id_autor) VALUES (1, 'Dom Casmurro', 1);
INSERT INTO Livros (id_livro, titulo_livro, id_autor) VALUES (2, 'Senhora', 1);
INSERT INTO Livros (id_livro, titulo_livro, id_autor) VALUES (3, 'Budapeste', 2);

-- Verificando os dados antes do DELETE
SELECT * FROM Autores;
SELECT * FROM Livros;

-- Tentando excluir um autor (deve falhar porque o autor tem livros associados)
DELETE FROM Autores WHERE id_autor = 1;

-- Verificando os dados após a tentativa de DELETE (A exclusão do autor não ocorrerá devido ao RESTRICT)
SELECT * FROM Livros;
```


RESTRICT

Autores

id_autor	nome_autor
----------	------------

1	Machado de Assis
---	------------------

2	Chico Buarque
---	---------------

Livros

id_livro	titulo_livro	id_autor
----------	--------------	----------

1	Dom Casmurro	1
---	--------------	---

2	Senhora	1
---	---------	---

3	Budapeste	2
---	-----------	---

SET NULL

Com o SET NULL, se um autor for excluído, o campo **id_autor** na tabela **Livros** será **atualizado** para **NULL** nos livros relacionados.

```
CREATE DATABASE db_exemplo_setnull;
USE db_exemplo_setnull;

-- Criando a tabela Autores
CREATE TABLE Autores (
    id_autor INT PRIMARY KEY,
    nome_autor VARCHAR(100)
);

-- Criando a tabela Livros com a chave estrangeira usando SET NULL
CREATE TABLE Livros (
    id_livro INT PRIMARY KEY,
    titulo_livro VARCHAR(100),
    id_autor INT,
    FOREIGN KEY (id_autor) REFERENCES Autores(id_autor) ON DELETE SET NULL -- SET NULL
);

-- Inserindo dados
INSERT INTO Autores (id_autor, nome_autor) VALUES (1, 'Machado de Assis');
INSERT INTO Autores (id_autor, nome_autor) VALUES (2, 'Chico Buarque');

INSERT INTO Livros (id_livro, titulo_livro, id_autor) VALUES (1, 'Dom Casmurro', 1);
INSERT INTO Livros (id_livro, titulo_livro, id_autor) VALUES (2, 'Senhora', 1);
INSERT INTO Livros (id_livro, titulo_livro, id_autor) VALUES (3, 'Budapeste', 2);

-- Verificando os dados antes do DELETE
SELECT * FROM Autores;
SELECT * FROM Livros;

-- Excluindo um autor (autor com id 1)
DELETE FROM Autores WHERE id_autor = 1;

-- Verificando os dados após o DELETE (O campo id_autor será NULL para os livros relacionados ao autor excluído)
SELECT * FROM Livros;
```

SET NULL

Autores	
id_autor	nome_autor
1	Machado de Assis
2	Chico Buarque

Livros		
id_livro	titulo_livro	id_autor
1	Dom Casmurro	NULL
2	Senhora	NULL
3	Budapeste	2